

Software Enginnering

2nd Sem

Aditya Kumar

Contents

Chapter 1	Introduction	Page 2
1.1	Getting Started	2
	Why it exists / The problem it solves? — 2	

Chapter 1

Introduction

1.1 Getting Started

Software engineering is built on the premise that as software projects move from small, individual efforts to large-scale team endeavours, intuition must be replaced by a systematic, disciplined, and quantifiable approach. One thing that helped me recently when looking at architectural decisions was Mall's focus on "**Phase Containment**"—the idea that catching a bug in the same phase it was born isn't just a good habit; it's an economic necessity to keep costs from growing exponentially.

Here is how Mall defines the fundamental pillars of the discipline.

Explain Like I'm 5

Imagine you want to build a tiny birdhouse. You can probably just grab some wood, a hammer, and wing it. If you mess up, it's easy to fix. But now imagine you are asked to build a 50-storey skyscraper. You can't just "wing it" anymore. You need exact blueprints, a schedule for when the cranes arrive, and a way to test if the floors are steady before you build the next one.

Software engineering is that "skyscraper plan" for computers. Because we can't see code like we see a building, we need special rules and documents to make sure the hundreds of people working on it don't get confused and that the "building" doesn't fall down when we're finished.

The Real Explanation

Software engineering is a systematic collection of program development practices and techniques derived from decades of research and practitioner experience. It is essentially an engineering approach to software development, moving away from the "artistic" or "craft" style used in smaller projects.

1.1.1 Why it exists / The problem it solves?

It exists to tackle the "Software Crisis," where projects were consistently delivered late, over budget, and full of defects.

1. **The Complexity Ceiling:** In an exploratory ("build and fix") style, the effort required to build a project grows **exponentially** with size.
2. **Cognitive Limitations:** Humans can only track about seven unrelated items at once. Software engineering uses principles like **Abstraction** (modelling) and **Decomposition** (divide and conquer) to keep project effort **linear** and manageable.

3. How it works fundamentally:

Mall structures the discipline around the **Software Life Cycle Model (SDLC)**, which describes the stages from initial request to final retirement.

- (a) **Ordered Activities:** Development follows a defined path: Feasibility, Requirements (SRS), Design, Coding, Testing, and Maintenance.
- (b) **Visibility:** Because software is "invisible," progress is measured through consistent, peer-reviewed documents at each stage.
- (c) **Project Management Precedence:** Effective planning follows a strict logical flow: you estimate **Size** (using LOC or Function Points) first, then derive **Effort**, then **Duration**, and finally **Cost**.
- (d) **Maintenance Dominance:** A critical realization in Mall's work is that roughly **60%** of a software product's life cycle effort is spent on maintenance after delivery.

Key Takeaways

1. **Engineering vs. Art:** Professional software development requires a "programming-in-the-large" mindset where team coordination and systematic methodology are more important than individual "heroic" coding.
2. **Functional Independence:** A good design is characterized by high **cohesion** (how focused a module is) and low **coupling** (how much modules depend on each other).
3. **Verification vs. Validation:** Verification checks if we are building the product "right" (conforming to the previous phase), while validation checks if we built the "right" product (meeting the customer's actual needs).

MISCONCEPTION ALERT

1. **If we're behind schedule, we can just add more people.** Mall highlights *Brooks' Law*: "Adding people to a late software project makes it later". This happens because new team members require training from the existing staff, and the communication overhead for the project increases non-linearly.
2. **Once the code is written, the hard part is over.** According to the effort distribution data, the coding phase only accounts for a relatively small portion of the total effort.
3. **Testing and Maintenance** are the most effort-intensive phases in the software life cycle.

OPTIONAL — Further Exploration Want to try something slightly better? To master the practical application of these principles, check out: * **Requirement Traceability:** Learning how to map every requirement bubble in a DFD to a specific module in a structure chart and eventually to a test case. * **COCOMO II:** Studying this heuristic model to see how "Cost Drivers" like analyst capability or required reliability mathematically impact project effort. * **Object-Oriented Analysis and Design (OOAD):** Transitioning from function-oriented design to an entity-centric view using UML.